



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

Departamento de Informática

PROYECTO



MERCALocal

Manual Técnico

Autor: Jimmy Christian Valens Chalá
Curso Académico: 2025 - 2026

ÍNDICE

1. INTRODUCCIÓN
 - 1.1. Funcionalidades implementadas
 - 1.2. Roles de usuario y permisos
2. ARQUITECTURA DE LA APLICACIÓN
 - 2.1. Stack tecnológico
 - 2.1.1. Backend — PHP 8.3 con PDO
 - 2.1.2. Frontend — HTML5, JavaScript Vanilla y Tailwind CSS
 - 2.1.3. Base de datos — MySQL 8.0
 - 2.1.4. Entorno de desarrollo
3. DOCUMENTACIÓN TÉCNICA (ANÁLISIS Y DESARROLLO)
 - 3.1. Arquitectura del código — Patrón MVC artesanal
 - 3.2. Sistema de enrutamiento (Routing)
 - 3.2.1. El Servidor Integrado de PHP como Enrutador Nativo
 - 3.2.2. El Front Controller (public/index.php)
 - 3.2.3. El Motor de enrutamiento (App/Core/Router)
 - 3.3. Seguridad y gestión de accesos (Middleware)
 - 3.4. Estructura de la Base de Datos - Modelo Lógico
 - 3.4.1. Relaciones del Modelo y Reglas de Cardinalidad
 - 3.5. Diagramas del sistema
 - 3.5.1. Diagrama Entidad-Relación Extendido
 - 3.5.2. Diagrama de Clases
 - 3.5.3. Diagrama de casos de Uso
 - 3.5.4. Diagrama de Secuencia 1
 - 3.5.5. Diagrama de Secuencia 2
4. PRUEBAS REALIZADAS
5. PROCESO DE DESPLIEGUE
 - 5.1. Clonar el repositorio
 - 5.2. Configurar las variables de entorno
 - 5.3. Generar el cargador automático de clases (Composer Autoload)
 - 5.4. Instalar y compilar las dependencias de frontend
 - 5.5. Crear e importar la base de datos
 - 5.6. Lanzar el Servidor Interno de PHP
 - 5.7. Verificar el despliegue del sistema
6. PROPUESTA DE MEJORAS
 - 6.1. Refactorización del sistema de vistas - Parciales unificados
 - 6.2. Integración de pasarela de pagos - Carrito funcional
 - 6.3. Módulo de reserva de citas y gestión de promociones
7. BIBLIOGRAFÍA Y WEBGRAFÍA
 - 7.1. Literatura Técnica y Arquitectura de Software
 - 7.2. Cursos de Formación y Contenido Audiovisual
 - 7.3. Documentación Oficial y Especificaciones Técnicas
 - 7.4. Seguridad Web y Buenas Prácticas

1. INTRODUCCIÓN

Mercalocal es una plataforma web de tipo marketplace y catálogo interactivo que he diseñado y desarrollado como Proyecto Final de Grado del ciclo formativo de Desarrollo de Aplicaciones Web. Su propósito fundamental es ofrecer una solución digital accesible y pragmática para la digitalización del comercio local, permitiéndoles exponer sus productos en un escaparate web unificado, visible para cualquier usuario con acceso a un navegador.

La motivación del proyecto surge de una necesidad real y creciente: la brecha digital que separa a los pequeños comercios y autónomos locales de sus potenciales clientes en el entorno online. Mientras las grandes plataformas de comercio electrónico dominan el mercado, los comercios de barrio como fruterías, talleres, peluquerías, tiendas de artesanía carecen en su gran mayoría de presencia digital propia. Mercalocal nace para cubrir ese vacío proporcionando una herramienta sencilla, centralizada y sin coste técnico para el comerciante, donde cada negocio local puede registrarse, configurar su perfil y publicar su catálogo de productos.

1.1. Funcionalidades implementadas

El desarrollo actual de Mercalocal constituye un Producto Mínimo Viable (MVP) completamente funcional que incluye las siguientes capacidades operativas:

Autenticación segura de usuarios. He implementado un sistema completo de registro e inicio de sesión que emplea el algoritmo de hashing *bcrypt* (a través de la función nativa *password_hash()* de PHP) para el almacenamiento seguro de contraseñas. El proceso de login verifica las credenciales contra la base de datos utilizando consultas preparadas PDO, regenera el identificador de sesión tras la autenticación exitosa mediante *session_regenerate_id(true)* previniendo ataques de fijación de sesión (*session fixation*), y almacena en la superglobal *\$_SESSION* los datos esenciales del usuario autenticado: su identificador, nombre, correo electrónico y rol asignado.

Registro y gestión del perfil de comercios locales. Cada usuario con rol de comerciante (*business*) puede crear y administrar su ficha de negocio, que incluye nombre comercial, descripción, categoría, dirección física, teléfono de contacto e imagen representativa. El acceso público al perfil de cada comercio se realiza de forma dinámica mediante URLs semánticas gestionadas por el enrutador del sistema (por ejemplo, *business/detail*). El sistema identifica y renderiza la información del comercio de manera segura en base a su identificador único (*id*) asociado en la base de datos MySQL, garantizando una carga limpia y directa de la vista correspondiente sin sobrecargar el servidor con consultas complejas. La subida de imágenes está validada por tipo MIME (JPEG, PNG, WebP) y se almacena en el directorio *public/uploads/businesses/*.

Gestión de un catálogo interactivo de productos. Cada comerciante gestiona de forma autónoma el inventario de su negocio a través de un panel CRUD (Crear, Leer, Actualizar, Eliminar) completo. Los productos se definen con nombre, descripción, precio, stock disponible, imagen y estado (*activo/inactivo*). El catálogo público renderiza exclusivamente los productos marcados como activos, permitiendo a los visitantes navegar por los comercios y sus ofertas sin necesidad de registro.

Sistema de carrito de compra y pedidos (parcial/simulado). El MVP incorpora un carrito de compra basado en sesión (`$_SESSION['cart']`) que permite a los clientes registrados agregar productos, actualizar cantidades, eliminar artículos y vaciar el carrito. Existe igualmente un flujo de confirmación de pedidos que persiste la transacción en base de datos (tablas *purchase* y *order_item*), con un sistema de estados (pendiente, preparando, listo, completado, cancelado). No obstante, es imprescindible aclarar que esta funcionalidad se encuentra en fase de simulación dentro del MVP: no existe integración con pasarela de pago real ni lógica de notificaciones al comerciante; se trata de un esqueleto funcional preparado para su evolución en futuras iteraciones.

1.2. Roles de usuario y permisos

El sistema de control de acceso se articula en torno a tres roles claramente diferenciados, almacenados como un campo *ENUM* en la tabla *user*:

Rol	Identificador	Capacidades
Administrador	<i>admin</i>	Acceso al panel de control global para la supervisión visual de los comercios registrados. Por seguridad y diseño del MVP, la gestión crítica de usuarios (como la eliminación o el cambio de roles) se realiza de forma directa y securizada desde la propia base de datos, evitando acciones accidentales desde la interfaz web.
Comerciante	<i>business</i>	Creación y edición de su ficha de negocio. Gestión CRUD completa de su catálogo de productos. Visualización y gestión de estados de los pedidos recibidos.
Cliente	<i>user</i>	Navegación libre por el catálogo público de comercios y productos. Gestión de su carrito de compra. Realización y seguimiento de pedidos (simulado en esta fase).

El acceso a cada sección de la aplicación está gobernado por el componente *Middleware* (detallado en la sección 3.3 de este manual), que intercepta cada petición HTTP y evalúa el rol del usuario en sesión antes de conceder o denegar el acceso a la ruta solicitada.

2. ARQUITECTURA DE LA APLICACIÓN

2.1. Stack tecnológico

He seleccionado deliberadamente un stack tecnológico ligero, directo y sin dependencias de frameworks pesados. Esta decisión responde a una filosofía de desarrollo que denomino enfoque *solopreneur*: maximizar el control absoluto sobre cada línea de código, minimizar la curva de aprendizaje asociada a la configuración de herramientas de terceros, y garantizar que el resultado final sea un producto comprensible, mantenible y extensible sin necesidad de dominar las convenciones internas de un framework ajeno.

2.1.1. Backend — PHP 8.3 con PDO

El motor de la lógica de servidor de Mercalocal es PHP en su versión 8.3, utilizado en su forma nativa —sin frameworks como Laravel, Symfony o CodeIgniter—. La elección de PHP puro no es una limitación sino una decisión arquitectónica deliberada:

- **Rendimiento y ligereza.** Al prescindir de la capa de abstracción y el bootstrap que impone un framework (carga del contenedor de dependencias, resolución del service provider, inicialización del ORM, etc.), cada petición HTTP recorre un camino de ejecución significativamente más corto. El Front Controller (*public/index.php*) actúa como punto de entrada único de **Mercalocal** ejecutando estrictamente tres pasos: carga el archivo de configuración del sistema, inicia la sesión mediante *session_start()* y delega el despacho de la URL al enrutador nativo (*Router.php*), el cual procesa la petición antes de dar paso al controlador de negocio. Esta austeridad se traduce en tiempos de respuesta consistentemente bajos y un consumo de memoria predecible.
- **Control absoluto del código.** Cada componente del sistema —el router, el middleware, la clase de conexión a base de datos, el motor de vistas— ha sido escrito manualmente para este proyecto. Esto me permite entender, depurar y modificar cualquier comportamiento del sistema sin necesidad de estudiar la documentación interna de un framework o navegar por miles de líneas de código de terceros.

- **Conexión segura a base de datos mediante PDO.** Todas las interacciones con MySQL se realizan a través de la extensión PDO (*PHP Data Objects*), configurada con los siguientes atributos de seguridad:
 - `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION`: los errores de base de datos se propagan como excepciones PHP, permitiendo su captura y manejo controlado.
 - `PDO::ATTR_EMULATE_PREPARES => false`: se desactivan las consultas preparadas emuladas, forzando al driver MySQL nativo a manejar la parametrización, lo cual constituye la defensa más robusta contra inyecciones SQL.
 - `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC`: los resultados se devuelven como arrays asociativos por defecto, simplificando el acceso a los datos en los controladores y vistas.
- **Aprovechamiento de PHP moderno:** El código aprovecha características eficientes introducidas en las versiones recientes de PHP, como el operador nullsafe (`?->`) para evitar errores por valores nulos, y las funciones nativas `str_starts_with()` y `str_contains()` para optimizar el análisis y emparejamiento de las URLs dentro del sistema de enrutamiento.

2.1.2. Frontend — HTML5, JavaScript Vanilla y Tailwind CSS

La capa de presentación se construye sobre tres pilares:

- **HTML5 semántico.** La estructura de todas las vistas emplea etiquetas semánticas del estándar HTML5 (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`) para garantizar la accesibilidad, la correcta indexación por motores de búsqueda y la claridad estructural del marcado.
- **JavaScript vanilla.** La interactividad del lado del cliente se implementa mediante JavaScript nativo, sin dependencia de librerías como jQuery ni frameworks reactivos como React o Vue. Esta decisión es coherente con la naturaleza del MVP: las interacciones requeridas (envío de formularios, animaciones de feedback, toggles de interfaz) no justifican la incorporación de una librería externa.
- **Tailwind CSS 3.4.** Para el diseño visual y la maquetación responsive, he adoptado el framework de utilidades Tailwind CSS. Su enfoque *utility-first* permite componer interfaces directamente en el marcado HTML mediante clases atómicas predefinidas (`flex`, `gap-4`, `rounded-lg`, `bg-primary-600`, `hover:shadow-xl`), eliminando la necesidad de escribir hojas de estilo CSS personalizadas para cada componente. En producción Tailwind genera un archivo CSS optimizado y purgado que contiene exclusivamente

las clases utilizadas en las vistas. La configuración personalizada del proyecto (*tailwind.config.js*) extiende la paleta de colores con una escala cromática *primary* basada en tonos verdes evocando frescura, proximidad y comercio local y define la tipografía *Inter* como fuente principal del sistema.

2.1.3. Base de datos — MySQL 8.0

El almacenamiento persistente de la aplicación recae sobre MySQL en su versión 8.0, empleando el motor de almacenamiento InnoDB para todas las tablas. La elección de InnoDB es fundamental porque garantiza:

- **Integridad referencial mediante claves foráneas.** Las relaciones entre tablas (*user* → *business* → *product* → *order_item*) se respaldan con restricciones *FOREIGN KEY* con política *ON DELETE CASCADE*, asegurando la consistencia de los datos ante eliminaciones.
- **Transacciones ACID.** InnoDB cumple con las propiedades ACID (*Atomicity, Consistency, Isolation, Durability*), lo que resulta imprescindible para operaciones como la creación de pedidos, donde se insertan registros simultáneamente en las tablas *order* y *order_item*.
- **Soporte completo de Unicode.** El charset *utf8mb4* con collation *utf8mb4_unicode_ci* permite almacenar correctamente cualquier carácter Unicode, incluyendo emojis y caracteres especiales del español (tildes, eñes).

2.1.4. Entorno de desarrollo

El desarrollo de Mercalocal se ha llevado a cabo íntegramente en un entorno Windows con las siguientes herramientas:

Herramienta	Versión / Detalle	Función
Sistema Operativo	Windows 11	Plataforma base de desarrollo
IDE	Visual Studio Code	Editor de código
Asistentes IA	GitHub Copilot	Asistencia contextual de codificación integrada en el IDE
Servidor local	Servidor integrado de PHP 8.3	Servidor web nativo levantado por consola para la ejecución ligera del MVP.

PHP	8.3.x	Intérprete del lenguaje de backend
MySQL	8.0.x	Sistema gestor de base de datos relacional
Node.js / npm	LTS	Ejecución del CLI de Tailwind CSS para compilación de estilos
Git	2.x	Sistema de control de versiones distribuido

3. DOCUMENTACIÓN TÉCNICA (ANÁLISIS Y DESARROLLO)

3.1. Arquitectura del código — Patrón MVC artesanal

He estructurado Mercalocal siguiendo el patrón arquitectónico Modelo-Vista-Controlador (MVC) implementado de forma artesanal y ligera, sin recurrir a implementaciones framework-dependientes. Esta separación de responsabilidades organiza el código en tres capas claramente delimitadas:

mercalocal/	
├── app/	
│ ├── Controllers/	(Controladores: lógica de negocio y gestión de peticiones)
│ ├── Core/	(Núcleo del sistema: Router.php, Middleware.php y Database.php)
│ └── Models/	(Modelos: consultas, abstracción y persistencia de datos con MySQL)
├── database/	(Ficheros de base de datos y scripts SQL de inicialización)
├── logs/	(Registro interno de errores y auditoría del sistema)
├── public/	
│ ├── css/	(Estilos e interfaz visual compilada con Tailwind CSS)
│ ├── img/	(Imágenes estáticas e iconografía de la aplicación)
│ ├── js/	(Scripts de comportamiento y soporte dinámico en el cliente)
│ └── uploads/	(Directorio de almacenamiento para imágenes subidas por usuarios)
├── resources/	
│ └── views/	(Vistas: plantillas HTML/PHP estructuradas de la interfaz)
└── config.php	(Archivo centralizado de configuración y constantes globales)

Capa Modelo (*app/Models/*)

Los modelos encapsulan el acceso a la base de datos. Cada modelo es una clase PHP con métodos estáticos que ejecutan las consultas SQL necesarias para una entidad concreta. No implementan un ORM ni un Active Record completo; su responsabilidad se limita a proporcionar una interfaz limpia de lectura y escritura sobre la tabla correspondiente, utilizando consultas preparadas PDO.

Capa Controlador (*app/Controllers/*)

Los controladores son clases PHP que reciben las peticiones HTTP ya despachadas por el router, ejecutan la lógica de negocio correspondiente (validaciones de entrada, interacciones con los modelos, gestión de sesión, subida de archivos) y determinan la respuesta: renderizar una vista, redirigir a otra ruta o devolver un código de error HTTP. Cada controlador agrupa los métodos relacionados con un dominio funcional.

Capa Vista (*resources/views/*)

Las vistas son archivos estructurados que combinan marcado HTML con código PHP dinámico para la presentación de datos. Reciben las variables directamente desde los controladores y se organizan de forma lógica en subdirectorios dentro de *resources/views/* según su funcionalidad (gestión de usuarios, administración, comercios o productos).

Para garantizar la seguridad de la plataforma, todas las vistas implementan la función *helper e()* un alias optimizado de *htmlspecialchars()* configurado con codificación UTF-8 y *ENT_QUOTES*. Esta función escapa de forma estricta cualquier salida de datos dinámicos introducidos por los usuarios, constituyendo la defensa principal del sistema frente a vulnerabilidades de tipo Cross-Site Scripting (XSS).

Capa Core (*app/Core/*)

El directorio *Core/* centraliza los componentes esenciales de la infraestructura que sostienen la arquitectura del proyecto:

- **Router.php:** Sistema nativo de enrutamiento encargado de capturar las peticiones HTTP y derivarlas al controlador correspondiente.
- **Middleware.php:** Capa intermedia encargada del control de acceso, verificación de sesiones y seguridad antes de la ejecución de la lógica del negocio.
- **Database.php:** Componente de infraestructura encargado de la conexión a MySQL vía PDO mediante el patrón de diseño **Singleton** (*Database::getInstance()*).

La clase *Database* mantiene una única instancia de la conexión PDO durante el ciclo de vida de la petición HTTP, optimizando el uso de recursos al evitar la apertura redundante de múltiples conexiones con el motor de base de datos. Toda la interacción con el modelo de datos se realiza a través de sentencias preparadas nativas de PDO, lo que anula cualquier riesgo de inyección SQL en la plataforma.

3.2. Sistema de enrutamiento (Routing)

El sistema de enrutamiento de Mercalocal se fundamenta en el patrón arquitectónico **Front Controller**: un punto de entrada único que centraliza la recepción y el despacho de todas las peticiones HTTP de la aplicación. En nuestro entorno de desarrollo, esto se logra de manera nativa sin necesidad de servidores de terceros ni archivos de configuración complejos.

3.2.1. El Servidor Integrado de PHP como Enrutador Nativo

Al prescindir de Apache (XAMPP/Laragon) y utilizar el **servidor web integrado de PHP** (*php -S localhost:port*), el comportamiento del Front Controller cambia de forma eficiente. El servidor integrado actúa por sí mismo como un enrutador básico: si la petición coincide con un archivo físico existente (como el CSS compilado de Tailwind o una imagen en *public/uploads*), el servidor lo sirve directamente. Si la ruta no existe físicamente, PHP redirige de forma automática toda la carga de la petición hacia el script de entrada principal.

3.2.2. El Front Controller (*public/index.php*)

El archivo *index.php* es el punto de entrada real de la aplicación. Su secuencia de ejecución es directa y lineal, optimizada para un desarrollo limpio:

- **Carga de Configuración** (*config.php*): Define las constantes globales del sistema, rutas de directorios y credenciales de acceso a la base de datos MySQL.
- **Autocarga de Clases** (*Autoload Nativo*): Utiliza una función nativa de registro (*spl_autoload_register*) basada en el estándar PSR-4 para cargar automáticamente las clases según su *namespace*, prescindiendo por completo de gestores de dependencias externos como Composer.
- **Gestión de Sesión** (*session_start()*): Inicializa el ciclo de vida de la sesión para el control de usuarios.
- **Definición y Despacho**: Registra las rutas disponibles mediante los métodos del enrutador y ejecuta el método *dispatch()* para resolver la petición actual.

3.2.3. El Motor de enrutamiento (*App/Core/Router*)

La clase *Router* gestiona el mapa de URLs de la aplicación mediante un sistema de emparejamiento directo y eficiente:

- **Registro de Rutas**: Los métodos *get()* y *post()* almacenan las URLs permitidas en un array interno indexado por el método HTTP correspondiente.
- **Resolución de la URI**: El motor limpia la URL entrante eliminando los parámetros de la query string (*?id=...*) y aislando la ruta limpia para compararla con las registradas.
- **Despacho**: Itera sobre las rutas. Al encontrar una coincidencia exacta, invoca al controlador y método declarados en el mapa de rutas, instanciando el controlador bajo demanda y ejecutando la lógica de negocio. Si la ruta no existe, establece un código de estado HTTP 404 y renderiza la vista de error correspondiente.

3.3. Seguridad y gestión de accesos (Middleware)

El archivo *app/Core/Middleware.php* actúa como un interceptor táctico de seguridad. Su propósito es evaluar el estado de la sesión del usuario y sus privilegios **antes** de que el controlador procese la petición.

En lugar de delegar el control de accesos a un cortafuegos externo, el enrutador invoca al método estático `Middleware::handle()` pasando la ruta solicitada.

El Middleware valida el flujo basándose en la información persistida en la superglobal `$_SESSION`:

- **Control de Invitados:** Si un usuario con sesión activa intenta forzar la URL de acceso (`/login` o `/registro`), el sistema lo intercepta e impide la duplicidad de sesiones.
- **Protección de Rutas Privadas:** Restringe el acceso a las vistas de usuarios, clientes y pasarelas de la plataforma si no existe un identificador de sesión válido, redirigiendo al formulario de autenticación.
- **Segmentación por Roles:** Evalúa de manera estricta el rol del usuario autenticado (Administrador, Comerciante o Cliente Local). Si un cliente intenta acceder a herramientas de gestión de comercios (`/products`), el middleware deniega el paso, protegiendo la integridad del negocio.
- **Control de Onboarding:** Valida que un comerciante haya completado el paso previo de registrar su establecimiento antes de permitirle la subida o edición de existencias en el catálogo local, garantizando un flujo lógico y secuencial en la base de datos.

3.4. Estructura de la Base de Datos - Modelo Lógico

El modelo lógico relacional de **Mercalocal** representa la estructura formal de persistencia de datos de la plataforma, detallando de manera estricta las claves primarias (PK), las claves foráneas (FK) y las reglas de integridad referencial. El esquema está constituido por un total de 9 tablas interrelacionadas:

USER (id, nombre, apellidos, telefono, email, password_hash, imagen, rol, created_at, updated_at, direccion)

ADDRESS (id, calle, numero, piso_puerta, codigo_postal, ciudad, provincia)

CATEGORY (id, nombre, tipo, parent_id)

- *FK* parent_id → CATEGORY(id) ON DELETE SET NULL

BUSINESS (id, user_id, id_categoria, nombre, descripcion, telefono, email, web, activo, logo_path, hero_path, created_at, updated_at)

- *FK* user_id → USER(id) ON DELETE CASCADE
- *FK* id_categoria → CATEGORY(id) ON DELETE SET NULL

BUSINESS_ADDRESS (business_id, address_id)

- *FK* business_id → BUSINESS(id) ON DELETE CASCADE
- *FK* address_id → ADDRESS(id) ON DELETE CASCADE

PRODUCT (id, business_id, category_id, nombre, descripcion, precio, unidad_medida, stock, imagen, activo, created_at, updated_at)

- *FK* business_id → BUSINESS(id) ON DELETE CASCADE
- *FK* category_id → CATEGORY(id) ON DELETE SET NULL

SCHEDULE (id, business_id, dia_semana, hora_apertura, hora_cierre)

- *FK* business_id → BUSINESS(id) ON DELETE CASCADE

PURCHASE (id, user_id, address_id, total, delivery_method, estado, created_at, updated_at)

- *FK* user_id → USER(id)
- *FK* address_id → ADDRESS(id)

ORDER_ITEM (id, purchase_id, product_id, cantidad, precio_unitario)

- *FK* purchase_id → PURCHASE(id) ON DELETE CASCADE
- *FK* product_id → PRODUCT(id) ON DELETE RESTRICT

3.4.1. Relaciones del Modelo y Reglas de Cardinalidad

Las relaciones entre las entidades de **Mercalocal** siguen un modelo relacional estrictamente normalizado en Tercera Forma Normal (3FN), garantizando la integridad referencial mediante las siguientes restricciones y cardinalidades:

- **USER 1:N BUSINESS (1 a Muchos):** Aunque la lógica de negocio actual restringe que un comerciante gestione un establecimiento, a nivel de base de datos la relación se define como un usuario con capacidad de administrar múltiples comercios si la plataforma escalase. Se vincula mediante *business.user_id* hacia *user.id* con borrado en cascada (*ON DELETE CASCADE*).
- **CATEGORY 1:N BUSINESS / PRODUCT (1 a Muchos):** Una categoría puede clasificar a múltiples negocios o a múltiples productos. Se establece mediante las claves foráneas *business.id_categoria* y *product.category_id* apuntando a *category.id*. Al aplicar *ON DELETE SET NULL*, si una categoría se elimina, los productos y comercios asociados no se borran, sino que quedan huérfanos de categoría temporalmente para proteger el catálogo.
- **CATEGORY 1:N CATEGORY (Relación Reflexiva / Jerárquica):** Permite estructurar subcategorías mediante el campo *category.parent_id*, el cual apunta recursivamente a la misma tabla *category.id*.
- **BUSINESS M:N ADDRESS (Muchos a Muchos):** La relación entre los comercios y las direcciones geográficas estructuradas se rompe mediante la tabla intermedia de ruptura *business_address*. Esto permite que un negocio cuente con una o varias sedes físicas y que una dirección física pueda albergar de forma independiente varios comercios (como locales en un mismo centro comercial o galerías). Se vincula mediante la clave primaria compuesta por (*business_id*, *address_id*).
- **BUSINESS 1:N PRODUCT (1 a Muchos):** Cada establecimiento comercial puede dar de alta y gestionar múltiples artículos en su catálogo. Relacionado a través de *product.business_id* hacia *business.id* con eliminación en cascada.
- **BUSINESS 1:N SCHEDULE (1 a Muchos):** Un comercio cuenta con múltiples registros de franjas horarias distribuidas por los días de la semana. Se mapea mediante *schedule.business_id* hacia *business.id*.
- **USER 1:N PURCHASE (1 a Muchos):** Cada cliente de la plataforma puede registrar un histórico de múltiples pedidos. Se formaliza mediante la clave foránea

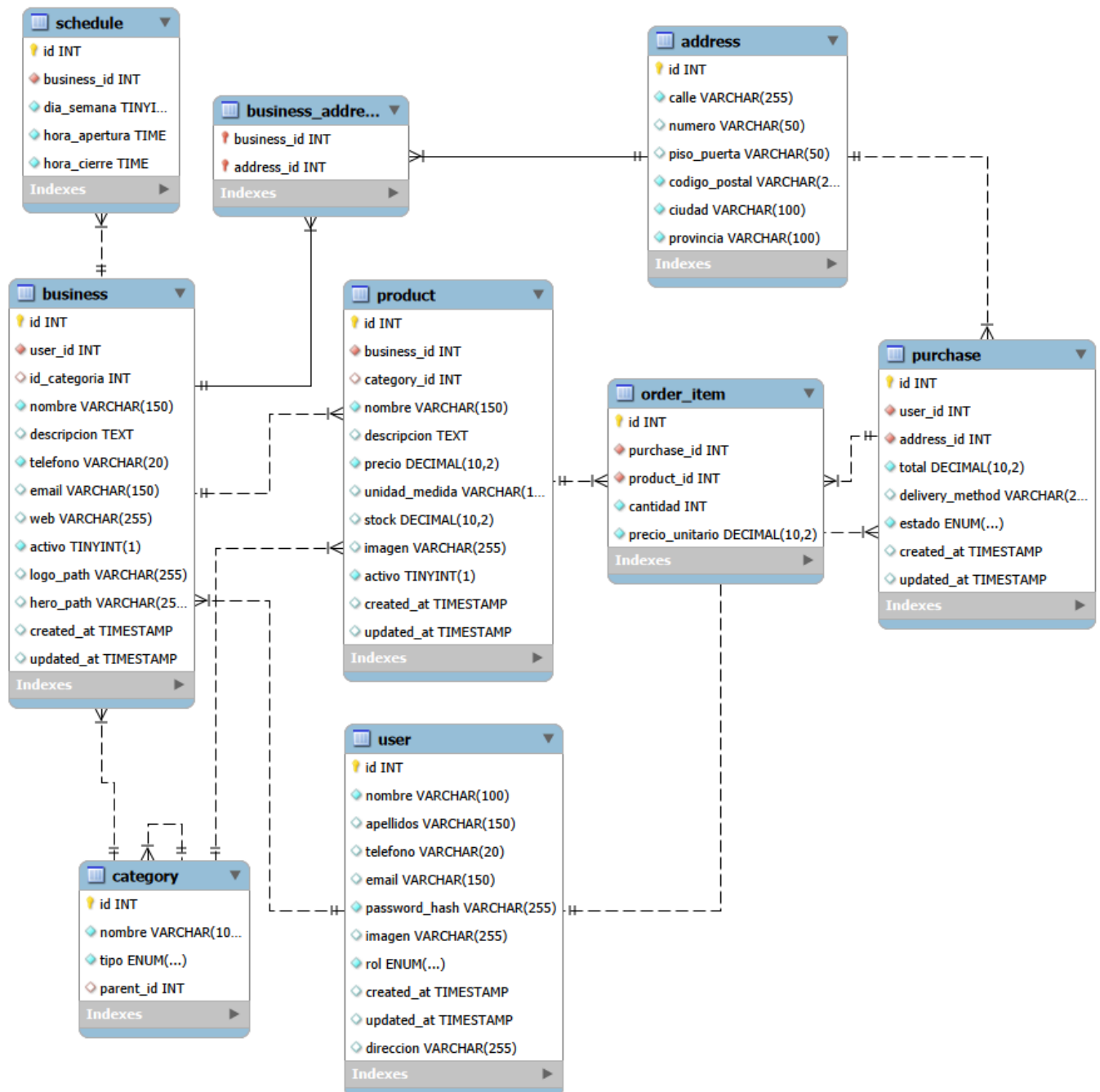
purchase.user_id que referencia a *user.id*. (Nota: La dirección residencial del usuario para la gestión de su perfil se almacena de forma simplificada como texto plano en el campo *user.direccion* para optimizar las consultas de la ficha de usuario).

- **ADDRESS 1:N PURCHASE (1 a Muchos):** Cada pedido se asocia de forma unívoca a una dirección física estructurada de entrega procedente de la tabla *address*. Se vincula mediante *purchase.address_id* apuntando a *address.id*.
- **PURCHASE 1:N ORDER_ITEM (1 a Muchos):** Cada documento o cabecera de pedido se desglosa en una o varias líneas de detalle independientes. Se establece mediante *order_item.purchase_id* referenciando a *purchase.id* con eliminación en cascada.
- **PRODUCT 1:N ORDER_ITEM (1 a Muchos):** Un mismo producto o servicio del catálogo general puede figurar en múltiples líneas de pedido históricas de diferentes clientes. Se controla mediante *order_item.product_id* apuntando a *product.id* bajo la restricción estricta de *ON DELETE RESTRICT* para impedir la eliminación de productos con actividad comercial pendiente.

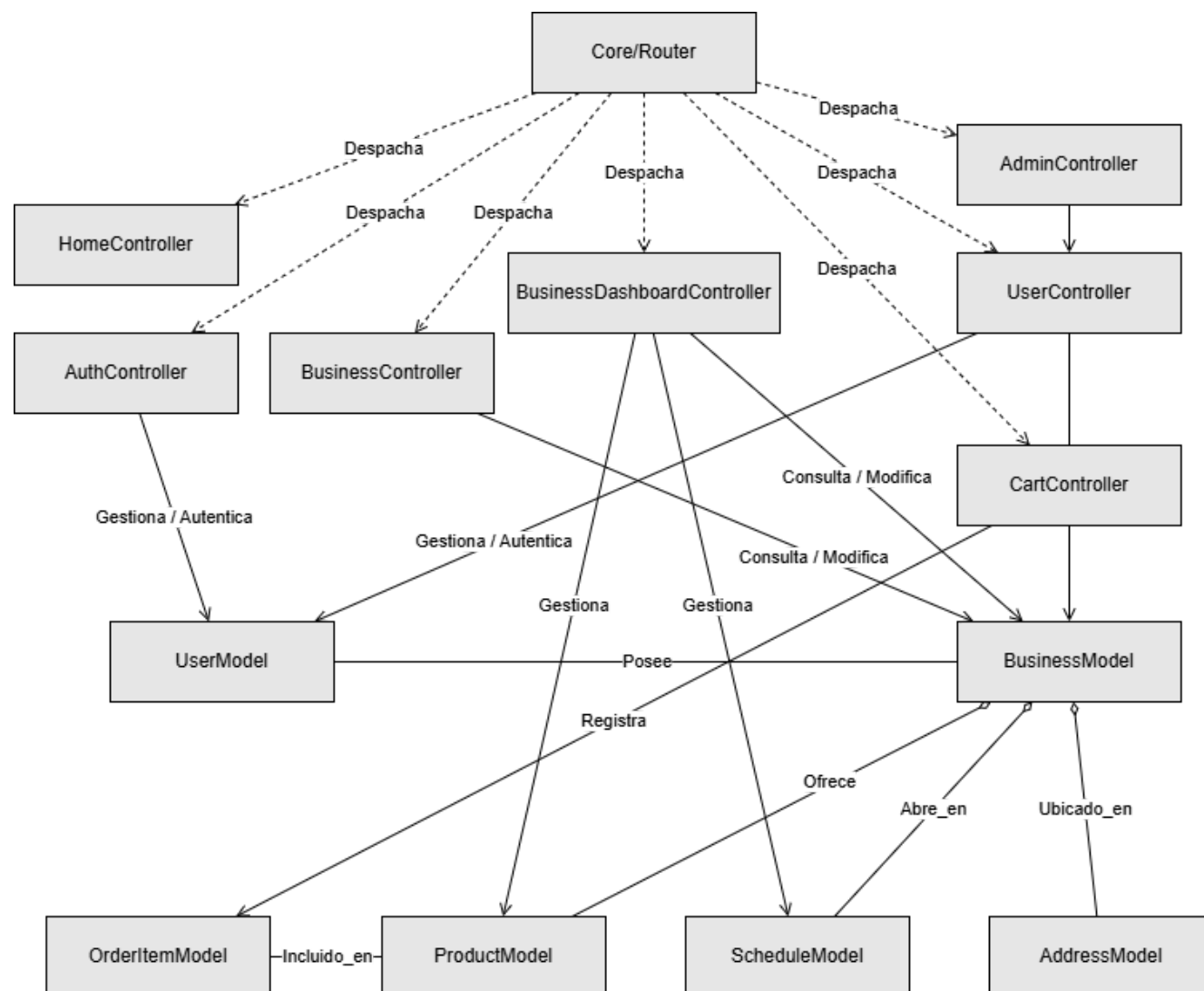
Todas las claves foráneas implementan la política *ON DELETE CASCADE*, lo que garantiza la eliminación en cascada de los registros dependientes cuando se borra un registro padre (por ejemplo, la eliminación de un usuario suprime automáticamente sus negocios, los productos de esos negocios, los pedidos asociados y las líneas de pedido correspondientes).

3.5. Diagramas del sistema

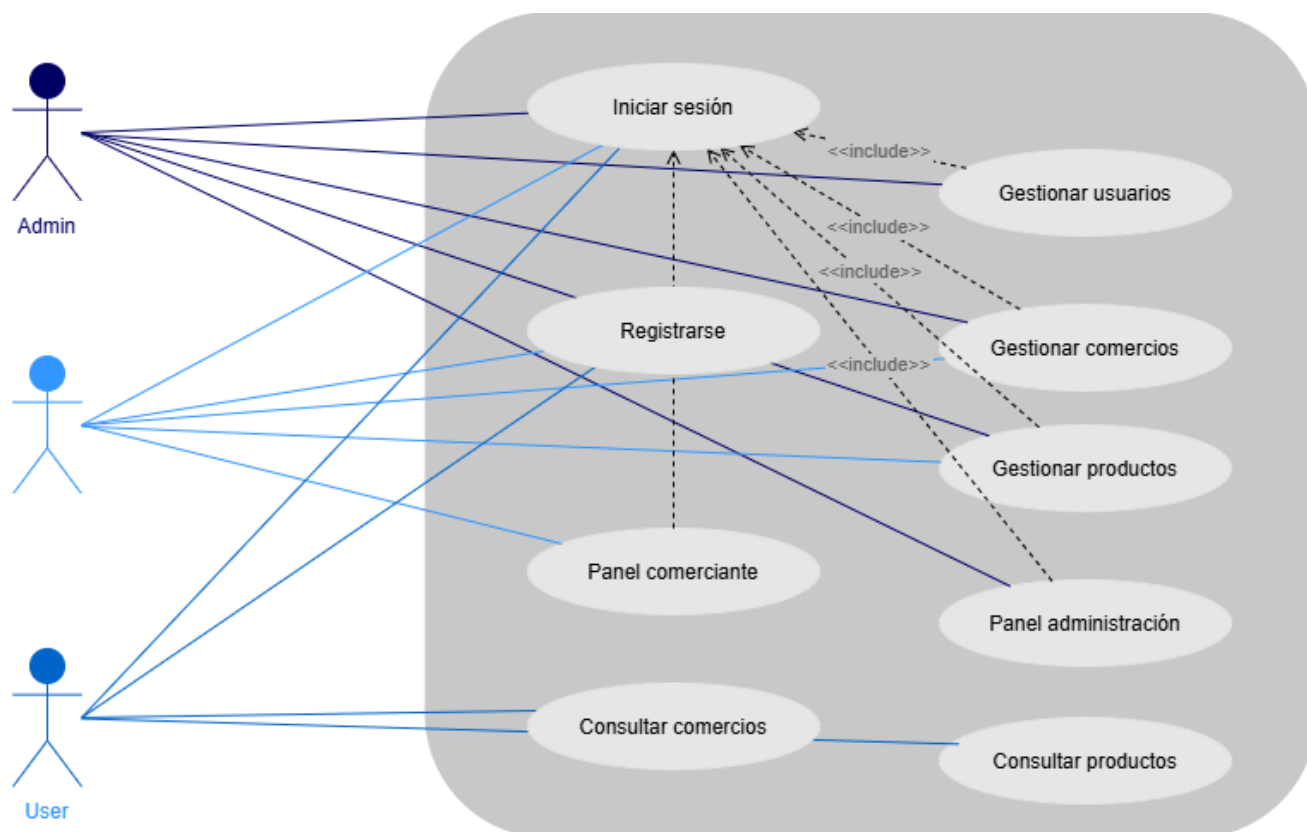
3.5.1. Diagrama Entidad-Relación Extendido



3.5.2. Diagrama de Clases

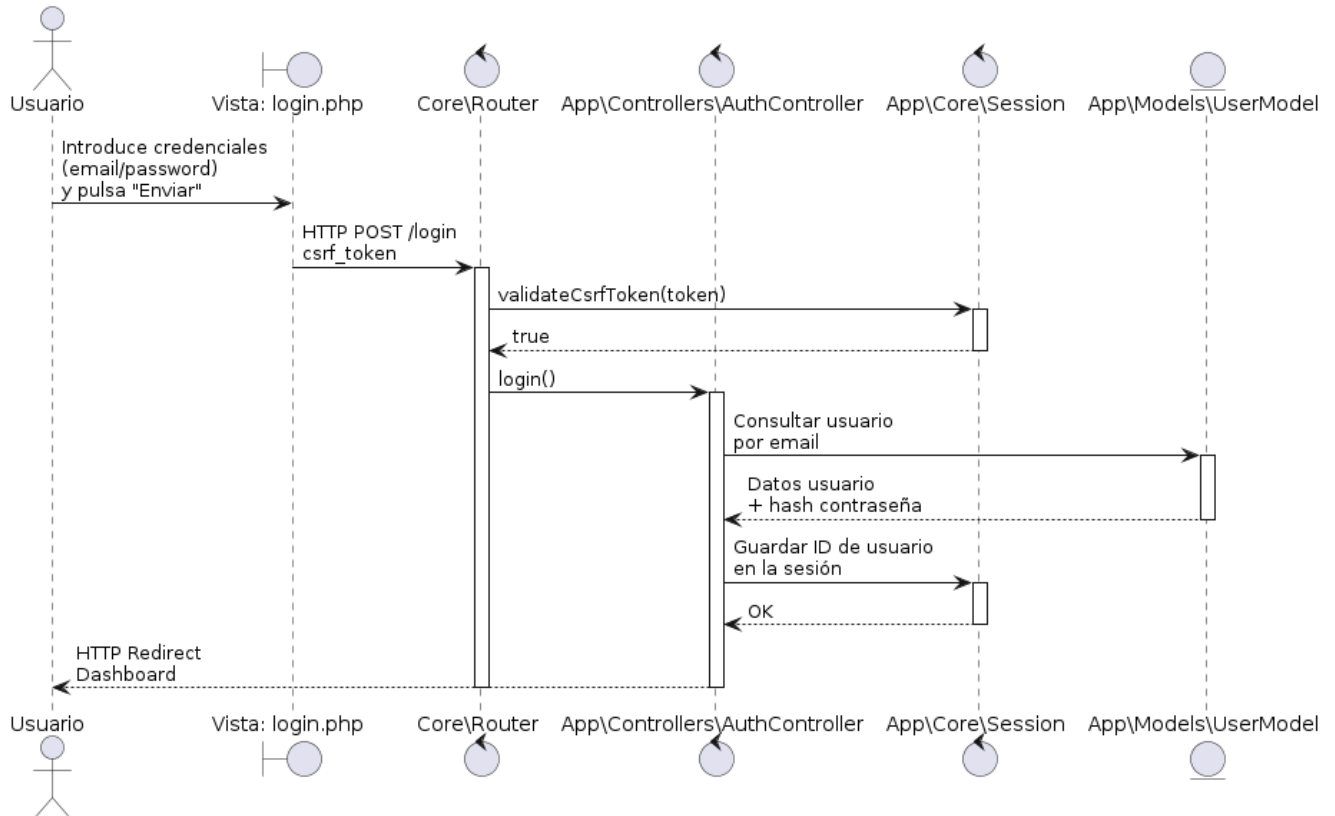


3.5.3. Diagrama de casos de Uso



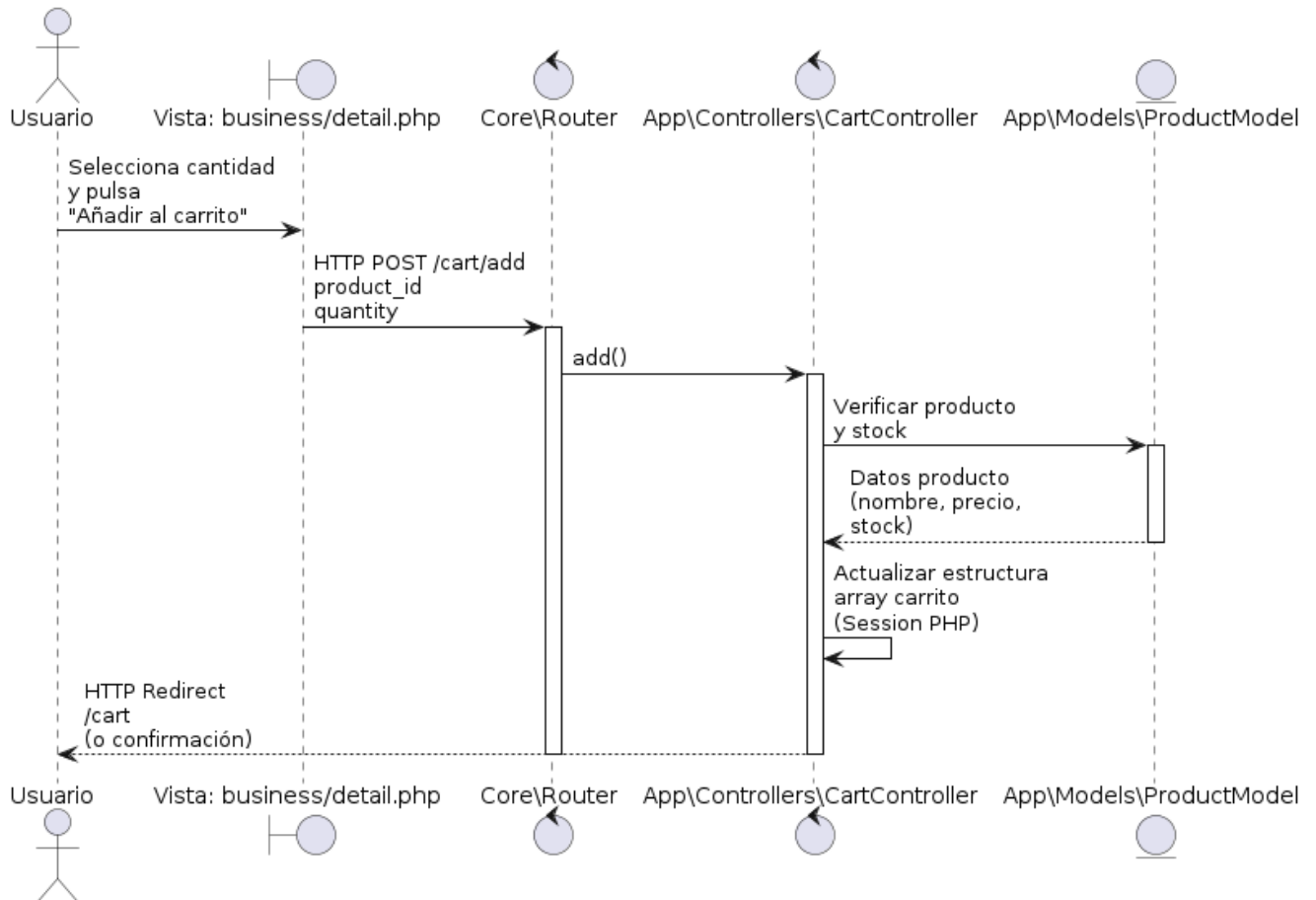
3.5.4. Diagrama de Secuencia 1

Diagrama de Secuencia - Flujo de Autenticación (Login)



3.5.5. Diagrama de Secuencia 2

Diagrama de Secuencia - Añadir Producto al Carrito



4. PRUEBAS REALIZADAS

La estrategia de verificación de Mercalocal se ha basado en pruebas funcionales manuales, ejecutadas directamente sobre la aplicación desplegada en el entorno de desarrollo local. Cada prueba se ha diseñado para validar una **funcionalidad específica del sistema** bajo condiciones controladas, documentando los datos de entrada, el resultado esperado y el resultado obtenido.

ID: PF-01 — Autenticación con credenciales válidas

- **Componente:** *AuthController*
- **Datos de entrada:** Formulario de login (*POST /login*): usuario y contraseña válidos.
- **Resultado esperado:** El controlador valida los datos, inicia la sesión mediante *Session::start()*, guarda al usuario en la sesión y redirige al panel correspondiente.
- **Resultado obtenido:** El usuario inicia sesión correctamente y es redirigido a su panel de control sin errores.
- **Estado:** Superada

ID: PF-02 — Autenticación con credenciales inválidas

- **Componente:** *AuthController*
- **Datos de entrada:** Formulario de login (*POST /login*): contraseña incorrecta.
- **Resultado esperado:** El controlador detecta el fallo, no genera variables de sesión, establece un mensaje flash de error y redirige de vuelta a */login*.
- **Resultado obtenido:** Se deniega el acceso, la sesión permanece vacía y se muestra el mensaje de error en el formulario.
- **Estado:** Superada

ID: PF-03 — Seguridad Global (Filtro CSRF)

- **Componente:** Controlador Frontal (*index.php*)
- **Datos de entrada:** Envío de cualquier formulario (*POST*) modificando o eliminando el campo *csrf_token*.
- **Resultado esperado:** El bloque de seguridad de *index.php* detecta que *Session::validateCsrfToken()* falla, aborta la petición, genera un mensaje flash de error y redirige al origen.
- **Resultado obtenido:** La petición es interceptada inmediatamente por el controlador frontal, denegando el procesamiento del formulario.
- **Estado:** Superada

ID: PF-04 — Enrutamiento Fijo

- **Componente:** *Core/Router*
- **Datos de entrada:** Acceso vía barra de direcciones del navegador a (*GET /businesses*).
- **Resultado esperado:** El enrutador analiza la URI, localiza la coincidencia exacta en su registro interno y ejecuta el método *BusinessController::index()*.
- **Resultado obtenido:** El catálogo público de comercios se renderiza correctamente, extrayendo y mostrando los datos desde la base de datos MySQL.
- **Estado:** Superada

ID: PF-05 — Enrutamiento Dinámico con Parámetros

- **Componente:** *Core/Router*
- **Datos de entrada:** Acceso vía barra de direcciones del navegador a (*GET /business/5*).
- **Resultado esperado:** El enrutador procesa la URI utilizando expresiones regulares, extrae el parámetro dinámico *{id}* (valor 5) y lo transfiere al método *BusinessController::detail(\$id)*.
- **Resultado obtenido:** Se despliega la ficha específica del comercio número 5 de forma aislada, sin interferir con el resto de rutas.
- **Estado:** Superada

ID: PF-06 — Control de Rutas Inexistentes (Error 404)

- **Componente:** *Core/Router*
- **Datos de entrada:** Acceso a una URL aleatoria no registrada (ej. *GET /ruta-inventada*).
- **Resultado esperado:** El método *dispatch()* recorre el mapa de rutas sin encontrar coincidencias, maneja la excepción, establece el código de estado HTTP 404 y carga la vista de error.
- **Resultado obtenido:** El navegador recibe correctamente el código de estado 404 en las cabeceras y se muestra la pantalla personalizada de "Página no encontrada".
- **Estado:** Superada

5. PROCESO DE DESPLIEGUE

A continuación, se detallan los pasos necesarios para poner en marcha la aplicación **Mercalocal** en un entorno local. El procedimiento asume una instalación funcional de PHP 8.3, MySQL y Node.js en el sistema.

5.1. Clonar el repositorio

Abrir la terminal en el directorio local de trabajo o proyectos y clonar el código fuente del sistema: :

```
git clone https://github.com/jimmyvalens/mercalocal.git  
cd mercalocal
```

Al finalizar, se obtiene la copia completa de la estructura de la aplicación en la máquina local (controladores, modelos, vistas, configuración de Tailwind y esquema SQL), quedando la terminal situada directamente en la raíz del proyecto.

5.2. Configurar las variables de entorno

Antes de inicializar cualquier proceso del sistema, se debe crear el archivo de entorno local copiando la plantilla de ejemplo:

```
copy .env.example .env
```

Acto seguido, se edita el archivo `.env` configurando los parámetros críticos del sistema:

```
# Configuración de entorno para la aplicación

# Base de Datos (Conexión)
DB_HOST=localhost
DB_USER=root
DB_PASS=root
DB_NAME=mercalocal_3

# Configuración del Sistema y Depuración
APP_ENV=development
DISPLAY_ERRORS=1
```

⚠ Nota para el despliegue en Producción: Al subir la aplicación al servidor definitivo en internet, es indispensable modificar los valores de la base de datos (`DB_HOST`, `DB_USER`, `DB_PASS` y `DB_NAME`) con las credenciales del servidor de producción. Asimismo, por motivos estrictos de seguridad, se debe cambiar `APP_ENV=production` y `DISPLAY_ERRORS=0` para evitar la exposición de trazas de código ante errores en el cliente.

5.3. Generar el cargador automático de clases (Composer Autoload)

Ejecutar el gestor de paquetes de PHP para inicializar el mapa de directorios del proyecto:

```
composer dump-autoload
```

Este comando no descarga librerías externas, sino que lee la configuración de la arquitectura en el archivo `composer.json` y genera el archivo crítico `vendor/autoload.php`. Este archivo es imprescindible para activar el estándar de organización **PSR-4**, permitiendo que el controlador frontal (`index.php`) localice y cargue automáticamente las clases reales (`App/Controllers/*`, `App/Models/*`, `App/Core/*`) mediante sus *namespaces* sin necesidad de escribir pesados *require* manuales.

5.4. Instalar y compilar las dependencias de frontend

Descargar el compilador de Tailwind CSS y procesar los estilos de la interfaz:

```
npm install  
npm run build
```

El comando *build* invoca al CLI de Tailwind CSS, el cual escanea todas tus vistas y plantillas PHP locales, detecta las clases de utilidad que has utilizado en las etiquetas HTML y genera el archivo CSS estático, optimizado y purgado en *public/css/style.css*. Para el desarrollo activo, se utiliza el comando *npm run dev* para activar el modo de escucha en tiempo real (*watch*), que recompila el archivo final automáticamente cada vez que guardas cambios en tus archivos PHP.

5.5. Crear e importar la base de datos

Desde la consola de MySQL o una herramienta gráfica (como phpMyAdmin), asegurar la existencia de la base de datos e importar el esquema relacional:

```
mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS  
mercalocal CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;"  
mysql -u root -p mercalocal < database/mercalocal.sql
```

Este script SQL automatiza la creación de las nueve tablas del modelo (*address*, *businesses*, *business_address*, *category*, *order_item*, *product*, *purchase*, *schedule*, *user*), establece las restricciones de claves foráneas.

5.6. Lanzar el Servidor Interno de PHP

Al seguir un enfoque ágil y orientado a un desarrollo independiente, Mercalocal no depende de servidores externos como Apache ni de archivos *.htaccess*. El sistema aprovecha el servidor web integrado de desarrollo que incluye PHP nativamente.

Para poner en marcha la aplicación, ejecuta el siguiente comando en la terminal desde la raíz del proyecto:

```
php -S localhost:8000 -t public
```

La bandera `-t public` le indica a PHP que la raíz del sitio web es la carpeta `public/`. Gracias a esto, el servidor protege el resto del código fuente y mapea automáticamente cualquier URL limpia (como `/login` o `/business/5`) enviándola directamente al controlador frontal `index.php`.

5.7. Verificar el despliegue del sistema

Abrir el navegador web y acceder directamente a la URL: `http://localhost:8000`. Realizar el chequeo de los siguientes puntos de control:

- **Estilos:** Confirmar que la interfaz carga correctamente el archivo `public/css/style.css` compilado por Tailwind y el diseño se muestra impecable.
- **Enrutamiento:** Comprobar que al navegar a `/login` o interactuar con el sistema, el enrutador nativo responde de inmediato sin añadir subcarpetas a la URL.
- **Persistencia:** Iniciar sesión con las credenciales del administrador por defecto para validar la conexión con MySQL mediante PDO.
- **Seguridad:** Intentar forzar el acceso a una ruta protegida escribiéndola directamente en la barra de direcciones para comprobar que el sistema bloquea la petición y redirige automáticamente al login.

6. PROPUESTA DE MEJORAS

El estado actual de **Mercalocal** constituye un MVP (Producto Mínimo Viable) completamente funcional y demostrable. Sin embargo, su evolución hacia un entorno de producción real requiere una serie de optimizaciones técnicas estructuradas en tres ejes prioritarios:

6.1. Refactorización del sistema de vistas — Parciales unificados

- **Problema identificado:** En la fase actual del desarrollo, el marcado HTML del menú lateral de navegación (*sidebar*) se repite de forma literal en cada vista de los paneles internos (`admin/dashboard.php`, `business/dashboard.php`, `user/dashboard`, etc.). Esto

viola el principio **DRY (Don't Repeat Yourself)** y genera un alto coste de mantenimiento: cualquier modificación en los enlaces o estilos debe replicarse manualmente en todos los archivos, con el riesgo inherente de introducir inconsistencias visuales.

- **Solución propuesta:** Implementar un sistema de componentes parciales (*partials*) utilizando la directiva nativa *require* de PHP. Se extraerá el menú a un archivo único en *views/partials/sidebar.php* que recibirá como contexto el rol del usuario en sesión para renderizar dinámicamente los accesos permitidos.

```
<!-- Inclusión limpia en cualquier vista del panel -->  
<?php require VIEW_PATH . '/partials/sidebar.php'; ?>
```

Esta refactorización se extenderá progresivamente a otros elementos repetitivos como las tarjetas de métricas, los componentes de mensajes *flash* (alertas de éxito/error) y los modales de confirmación.

6.2. Integración de pasarela de pagos - Carrito funcional

- **Problema identificado:** El sistema de carrito actual opera bajo una simulación lógica: el cliente confirma el pedido y este se registra en la base de datos con estado *PENDIENTE*, pero no existe una validación de transacciones económicas reales.
- **Solución propuesta:** Integrar una pasarela de pagos profesional mediante dos alternativas complementarias para el entorno web:
 - **Stripe:** Para la gestión automatizada de pagos con tarjeta de crédito/débito. Su SDK para PHP permite implementar *Stripe Checkout Sessions*, delegando la pasarela en un formulario seguro en la nube y eximiendo al servidor propio de almacenar datos sensibles (cumpliendo el estándar PCI DSS).
 - **Redsys (con soporte Bizum):** Fundamental para el ecosistema comercial español. Al tratarse de un marketplace local, la integración de Redsys es clave para ofrecer compatibilidad con las tarjetas bancarias nacionales y, sobre todo, habilitar el pago instantáneo vía **Bizum**, lo que aportará la máxima confianza y tasa de conversión al usuario final.
- **Flujo arquitectónico:** La implementación requerirá la creación de un *PaymentController* y la reestructuración del proceso de compra: el pedido se persistirá inicialmente en la base de datos como *pending_payment* y, tras recibir la confirmación

asíncrona y segura mediante **Webhooks** desde la API de la pasarela, el sistema actualizará automáticamente el estado a *paid* para notificar al comerciante.

6.3. Módulo de reserva de citas y gestión de promociones

- **Problema identificado:** En su concepción inicial, Mercalocal está limitado a la exposición de un catálogo de productos físicos. Esto excluye a una proporción significativa del tejido empresarial local, constituido por negocios de servicios (peluquerías, centros de estética, talleres mecánicos o academias) cuya oferta se gestiona mediante cita previa.
- **Solución propuesta:** Desarrollar dos submódulos de negocio complementarios en el núcleo de la aplicación:
 - **Sistema de reservas:** Una interfaz con un calendario interactivo para que los comercios de servicios definan sus jornadas y franjas horarias disponibles, permitiendo al cliente reservar citas en tiempo real. Esto implicará expandir la base de datos con nuevas tablas relacionales (*time_slots* y *appointments*), desarrollar un *AppointmentController* e integrar una librería JavaScript ligera en el *frontend* como *FullCalendar*.
 - **Motor de promociones:** Permitir a los vendedores crear ofertas temporales (descuentos porcentuales, códigos promocionales o campañas 2x1) vinculadas a un rango de fechas. Requerirá el diseño de una tabla *promotions* y la implementación de lógica de negocio nativa en PHP para aplicar automáticamente estas rebajas tanto en el catálogo público como en el cálculo dinámico de los totales del carrito.

7. BIBLIOGRAFÍA Y WEBGRAFÍA

A continuación se relacionan las fuentes documentales, técnicas y normativas consultadas durante el diseño, desarrollo y documentación del proyecto Mercalocal:

7.1. Literatura Técnica y Arquitectura de Software

- **Fowler, M.** (2003). *Patterns of Enterprise Application Architecture*. Addison-Wesley. (Referencia para la implementación de los patrones arquitectónicos MVC y Front Controller en la estructura del proyecto).
- **Gamma, E., Helm, R., Johnson, R. y Vlissides, J.** (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Consulta para el

diseño del patrón Singleton en la conexión a la base de datos y Front Controller en el enrutamiento).

- **Martin, R. C.** (2009). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall. (Filosofía de desarrollo aplicada en el proyecto para garantizar la legibilidad, nomenclatura semántica y mantenimiento del código limpio).

7.2. Cursos de Formación y Contenido Audiovisual

- **De la Torre, J. P.** (2025). *Desarrollo Web Completo con HTML5, CSS3, JS, AJAX, PHP y MySQL* [Curso en línea]. Udemey. Disponible en: <https://www.udemy.com/>
- **YouTube.** (s.f.). *Consultas de soporte técnico, tutoriales de integración de herramientas y buenas prácticas de desarrollo independiente* [Plataforma de vídeo]. Disponible en: <https://www.youtube.com/>

7.3. Documentación Oficial y Especificaciones Técnicas

- **Composer.** (s.f.). *Dependency Manager for PHP - Documentation*. Disponible en: <https://getcomposer.org/doc/>
- **PHP Framework Interop Group (PHP-FIG).** (s.f.). *PSR-4: Autoloader Standard*. Estándar de la industria utilizado para la resolución automática de namespaces en el proyecto. Disponible en: <https://www.php-fig.org/psr/psr-4/>
- **Tailwind Labs.** (2024). *Tailwind CSS Documentation*. Guías de uso y configuración del compilador de utilidades CSS. Disponible en: <https://tailwindcss.com/docs>
- **The PHP Group.** (2024). *PHP: Manual de PHP (Core, PDO Extension & password_hash)*. Documentación oficial del lenguaje para el acceso seguro a bases de datos mediante PDO y funciones de hashing criptográfico para contraseñas. Disponible en: <https://www.php.net/manual/es/>
- **The Twelve-Factor App.** (s.f.). *Metodología para el desarrollo de aplicaciones web y gestión de variables de entorno (.env)*. Disponible en: <https://12factor.net/es/>

7.4. Seguridad Web y Buenas Prácticas

- **OWASP Foundation.** (2024). *OWASP Top Ten Web Application Security Risks*. Guía de referencia sobre vulnerabilidades críticas (Inyección SQL, XSS, fijación de sesión) cuyas contramedidas nativas han sido implementadas en el código de Mercalocal. Disponible en: <https://owasp.org/www-project-top-ten/>